

oKAI - Team25

oKAI - Team25 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A top-down endless survival bullet-hell for casual mobile players, built around **drawing symbols on the touchscreen to cast spells** and surviving endless enemy waves. Defeating enemies grants new spells, so the player adapts their strategy as the run progresses; 1–4 players can team up. Named technical features: a custom C++ **gesture-recognition engine** (geometric template matching — \$1 Unistroke / \$Q — with a multi-stage preprocessing pipeline: resampling to a fixed point count, rotational normalization, scale-invariant bounding-box scaling, centroid translation; Euclidean scoring against templates; target >90% recognition within a 100 ms latency window); **server-authoritative C++ multiplayer** for 1–4 players over LAN or cloud (clients send inputs/cast requests, the server owns players, enemies, spells, damage, drops and progression, sends periodic world snapshots; clients interpolate remote entities and use client-side prediction + server reconciliation for the local player; RTT/loss/bandwidth diagnostics; tested under simulated latency/jitter/loss); a **data-driven ECS** (spell/enemy/player attributes from external files, runtime add/remove/replace of components, systems parallelizable); and a **robust modular spell system** (spells do more than one-shot damage; fast authoring of unique effects). Core requirements name an external data source, a stable-30fps C++ loop (20+ enemies, 5+ spell objects), an event/snapshot application protocol, CI, JSON+XML data-driven architecture, a dedicated C++ authoritative back-end, and tools: an ECS diagnostics logger, an in-engine gameplay-telemetry API, an in-system game editor with developer diagnostics, and a **custom gesture-template editor** (draw a symbol, export as XML) to add/refine spell recognition.

Development environment: **Native Android**. Design features: gesture-based spellcasting (the primary mechanic), ≥ 5 spell types, ≥ 3 enemy types (Basic / Ranged / Tank), and ≥ 1 boss.

References:

- Vampire Survivors on Steam (the endless-survival "be the bullet hell" loop — defeat waves, collect new weapons/abilities each run, snowball your build; now with up to-4-player co-op — the survival-and-adapt loop precedent)
- Magicka on Steam (a dynamic spell-casting action game where the spell **input itself** is the mechanic, with up-to-four-player co-op and a wave-survival challenge mode — the input-driven-magic + co-op precedent)
- Arx Fatalis — Wikipedia (spells cast by **drawing runes with mouse gestures** that must be drawn correctly to fire — the direct gesture-recognition-to-cast precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	oKAI
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the stated dev environment; device: Pixel 9a) — the touchscreen gesture-drawing mechanic is native-touch-first; the Native PC "desktop diagnostic" build is the development/tools vehicle (ECS diagnostics, telemetry, editors). **Extended:** a Linux server the team sets up by M2 (full use T2) hosting the dedicated C++ authoritative game server for cloud lobbies + an external database (Firebase or a server-side store) for player/enemy/spell data.

Front end: A top-down 2D bullet-hell renderer (20+ enemies + 5+ spell objects on screen at a stable 30 fps), the live **gesture-drawing surface**

(stroke capture + trail feedback + recognized-spell cue), the spell/HUD overlays and the lobby screens. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES drawn in a GLSurfaceView for the realtime 2D; Web/desktop — SDL2/3 + OpenGL ES for the PC diagnostic build.

Back end: The dedicated ****C++ authoritative server**** (LAN or cloud) running the gameplay simulation for 1–4 players — validating inputs and cast requests, simulating players/enemies/spells, resolving damage and server-random loot drops, owning progression, and distributing snapshots + reliable gameplay events — plus the external data store for spell/enemy/player data.

Languages: C++ (engine + server).

Backend scope: Local now, server-ready (T2) — the authoritative server + external DB seam set up & basic-tested on the Linux host by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing on two fronts — the fixed-timestep gameplay simulation (20+ enemies + 5+ spell objects at a stable 30 fps, server-authoritative) **and** the real-time **gesture-recognition service** (resample → normalize rotation → scale → translate → Euclidean-score against templates, targeting >90% accuracy within 100 ms). This is the defining performant-C++ requirement.
- *Data-driven:* spell, enemy and player attributes load from JSON; gesture templates are XML point-sets authored by the gesture-template editor; the ECS adds/removes/replaces components at runtime (e.g. a `FlyingComponent` mid-run) so new spells and effects are authored as data, not code.
- *2D:* top-down sprite/particle rendering of the bullet-hell (enemies, spells, drops, the boss) plus the gesture stroke/feedback layer — custom-rendered. *3D:* not needed. *Networking:* server-authoritative snapshots + reliable events for 1–4 players, client prediction/reconciliation, RTT/loss/bandwidth diagnostics, tested under simulated latency/jitter/loss.
- *Custom tool:* the **gesture-template editor** (draw → export XML → registered as a castable spell), plus the in-system game editor and the ECS diagnostics logger + gameplay-telemetry API (the "desktop diagnostic core" build).

- *External DB*: player/enemy/spell/progression data. *Auth* »: player accounts. *Analytics* »: the gameplay-telemetry API (quit-points, retry counts, ability usage, tutorial time) — optional/T2 but explicitly designed for.

Proposed new pool goal (requires instructor approval): **Gesture Recognition**

Engine — the \$1/\$Q template-matching pipeline as its own graded CS goal (currently folded into M1G1 ★ core logic + tested in M1G3 ★). If approved it would append to the Games pool (e.g. G17) and become the natural home for this signature system; the three picks below remain from the existing pool.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep loop (C++17/20) running the survival simulation and hosting the ****gesture-recognition core**** (stroke capture → resample → normalize → score); CMake, Git.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the gesture engine and M1G8 enemy AI run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** A runtime ECS in C++ with spell/enemy/player attributes in JSON and gesture templates in XML, hot-reloadable; components add/remove/replace at runtime (the ``FlyingComponent`` example); systems operate on component sets and are parallelizable.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M2G2 (the editors edit this data), M2G13 (the modular spell system), M1G8 (enemy data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays + the ECS diagnostics logger, Tracy profiling to hold 30 fps with 20+ enemies and 5+ spell objects, Catch2 tests for ****gesture recognition**** (accuracy/latency), spell casting and data loading, gated by a GitHub Actions Windows+Android matrix with clang-format/clang-tidy.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening incl. networking tests); the 30 fps / 100 ms budgets M2G1 validates on-device.
- **G4 2D Graphics** — the top-down bullet-hell renderer and the gesture stroke/feedback layer are the whole read of the game.

Tech & dependencies

- **Tech: Web build:** Sprite/particle renderer (enemies, spells, drops, boss), camera, batching for 20+ enemies + 5+ spells at 30 fps, and a stroke/trail overlay for the gesture; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType HUD. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (advanced spell VFX); every visible frame of the demo.
- **G8 Enemy AI & Pathfinding** — the ≥ 3 enemy types (Basic / Ranged / Tank) and the groundwork for the boss.

Tech & dependencies

- **Tech:** Finite-state machines + A*/steering for the enemy types, ranged-vs-melee-vs-tank behaviours and swarm movement toward the player, tuned per type from data (custom C++).
- **Prerequisites:** G1 (tick), G2 (enemy data).
- **Feeds into:** M2G10 (advanced AI + the boss); the endless-wave survival loop.
- **G11 Networking Foundation** — the server-authoritative 1–4-player multiplayer and its protocol.

Tech & dependencies

- **Tech: Web build:** Sockets (Winsock2/BSD, or ENet/SDL_net) between the C++ authoritative server and clients, a lobby/connection manager for up to 4 clients, a protocol separating periodic snapshots from reliable gameplay events, sequence numbers/timestamps to reject stale packets. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop the server simulates), G2 (state to serialize).

- **Feeds into:** M2G7 (match sync, prediction & reconciliation); the co-op feature.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling to hold 30 fps + the 100 ms gesture budget, ASan on MSVC, memory pools for enemies/spells/particles, and validation on-device (Pixel 9a); the Linux server (authoritative game server + external DB) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable and a stable frame/latency budget under full load.

- **G2 ★ Content Editor — Core — the **gesture-template editor** (draw → export XML → register a castable spell) plus the in-system game editor with developer diagnostics.**

Tech & dependencies

- **Tech:** Dear ImGui editor: draw/capture a gesture, resample/preview it, save the template as XML, and edit spell/enemy attributes as JSON, with round-trip serialization, live preview and undo/redo; developer-diagnostics readouts (import time, validation failures, hot-reload duration).
- **Libraries/Software:** Dear ImGui, nlohmann/json + a lightweight XML writer (e.g. pugixml), the project's SDL2/3/OpenGL ES renderer (live preview).
- **Prerequisites:** G2 (attribute/template data model), M1G4 (rendering for preview).
- **Feeds into:** M2G6-class advanced tooling; new/refined spell recognition; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, unit/integration tests

(gesture recognition, spell casting, data loading) + networking validation (serialization, server-side input validation, connect/disconnect, multi-client tests under simulated latency/jitter/loss), asset/scene validators, and artifact upload. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every oKAI goal.

- **G7 Networking — Match Sync & Prediction** — authoritative match state with client prediction and server reconciliation.

Tech & dependencies

- **Tech:** Server-authoritative snapshots + delta over the wire, interpolation of remote players/enemies, client-side prediction + reconciliation for the local player, and RTT/packet-loss/bandwidth/update-rate diagnostics.
- **Prerequisites:** M1G11 (networking foundation), M2G1 (optimization).
- **Feeds into:** Smooth 1–4-player co-op at showcase, validated under simulated latency/jitter/loss.

- **G10 AI — Advanced** — the boss encounter and richer enemy behaviour tiers.

Tech & dependencies

- **Tech:** Behaviour trees for the boss (phases/patterns), difficulty tiers and coordination across the Basic/Ranged/Tank mix, tuned from data (custom C++).
- **Prerequisites:** M1G8 (enemy AI foundation).
- **Feeds into:** The ≥ 1 boss encounter and the escalating-wave challenge at showcase.

- **G13 Gameplay Systems II** — the **robust, modular spell system**: spells do more than a single instance of damage, and new effects are fast to author.

Tech & dependencies

- **Tech:** A composable spell/effect system over the ECS (status effects, projectiles, areas, modifiers) so unique spells compose from reusable effect components; balancing data driving the ≥ 5 spell types.
- **Prerequisites:** M1G2 (ECS), M2G2 (editor authors spell data).
- **Feeds into:** The spell-arsenal showcase; T2 spell expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — bullet-hell reads by its **VFX and enemy variety**, so two artists carry the spell effects, the ≥ 3 enemy types + boss, the player and the gesture/HUD art; a single designer owns the gesture-spellcasting feel, the ≥ 5 -spell arsenal, the enemy/boss behaviours and the wave/run progression (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the spell/enemy **systems and multiplayer balance** outgrow one designer, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* a custom gesture-recognition engine + server-authoritative netcode + ECS + modular spells + ≥ 5 spells + ≥ 3 enemies + a boss + the editor suite is a very large surface — treat single-player survival with the gesture engine, ~ 3 spells, the 3 enemy types and the gesture-template editor as the M1/M2 floor, with co-op multiplayer, the boss and the full arsenal as the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the gesture-spellcasting survival loop, the ≥ 5 spells, ≥ 3 enemies + boss, and the adapt-your-build progression; primary benchmark e.g. Vampire Survivors, with a Singapore benchmark for the local-awareness criterion)
- M1D4 Game Feel & Touch Interaction Specification (the draw-to-cast feel — stroke forgiveness, recognition thresholds, cast feedback timing — authored as tunable parameters for the CS team, THE differentiator)
- M1D6 Systems & Economy Design (the ≥ 5 spells, the 3 enemy types, drops/loot and the run economy — every system with a source, a sink and a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (spell/enemy/drop balance and boss difficulty via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the bullet-hell HUD, the spell/gesture cues and the lobby, so the screen stays legible under 20+ enemies.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — HUD, spell feedback, lobby and results navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD + gesture feedback tested with casual players on the Pixel 9a.
- **Art M1:** 2 artists justified (VFX-heavy bullet-hell + enemy/boss roster)
 - M1A1 ★ Art Bible / Proof of Concept (the top-down visual direction — palette, readability under bullet-hell density, spell-language)
 - M1A2 Character Concepts & Model Sheets (the player, the 3 enemy types and the boss)
 - M1A3 Environment, Props & VFX Concepts (arenas, drops, and the spell-effect visual language)
 - M1A4 UI Art Concepts & Typography (gesture HUD, spell icons, the drawn-symbol vocabulary)
 - M1A5 Animation Foundations (enemy/boss/cast key poses and cycles)
 - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (enemies, the boss, cast reactions)
 - M2A3 Environment Alive — Ambient Animation & Layering
 - M2A4 UI Art — States & Feedback (recognized / mis-drawn / on-cooldown spell states)
 - M2A5 VFX — Critical Effects & Transitions (the spell effects — the game's core read — plus hit/death/boss VFX)
 - M2A6 Cutscenes & Storyboards (intro / boss / victory beats)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (tense

survival BGM, spell-cast, spell-hit, enemy and UI cues), suitable and correctly triggered.

- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for many overlapping spell/enemy SFX.

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (survival BGM + per-spell cast/impact, enemy, boss, pickup and UI SFX, ≥ 6 types), scaling cleanly with bullet-hell density.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct cue per recognized spell).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the gesture-template editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*